Informatics II, Spring 2026, Exercise 3

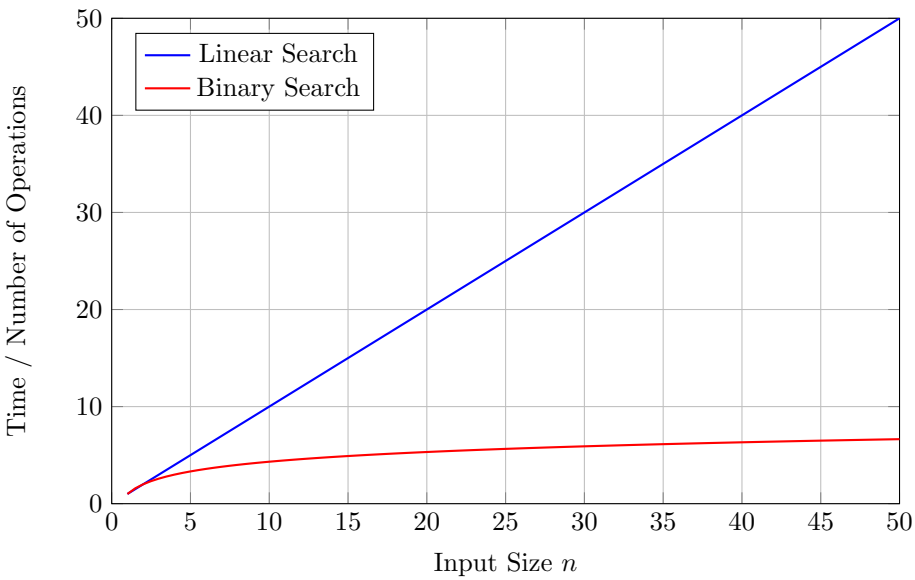
Publication of exercise: March 9, 2026
Publication of solution: March 16, 2026
Exercise classes: March 16 - March 20, 2026

Task 1 (Easy)

Consider an array A with n distinct integers that are sorted in an ascending order and an integer t.

a) The C funtion `linear_search` traverses the integers in A, one after another, from the beginning. If t is found in A `linear_search` returns 1, otherwise 0. Referencing the pseudocode for binary search in SL02, implement the C funtion `binary_search`(int A[], int n, int t) that employs binary search to find integer t in A. If t is found in A `binary_search` returns 1, otherwise 0. Complete the C funtion `binary_search`(int A[], int n, int t) in task1.c file.

b) Assuming the plot below displays the average runtime for Linear and Binary Search. Explain the differences in runtime. What is the worst case complexity of both algorithms?



Runtime comparison of linear search and binary search as a function of input size n. Please note that the displayed values have been mocked and where not measured.

c) Run your implementation of `binary_search` against the `linear_search` using the provided `compare_binary_and_linear()`. Adjust the parameters as desired. Does your implementation follow the same behavior as in the figure above?

Task 2 (Medium)

In this task you are given multiple code snippets to analyse in regards to the runtime. Feel free to reuse previously calculated results in future tasks of this exercise file.

a) For each of the following loops decide, how many times each line of code is executed.

i)

```
for i = n to 1 by −1 do
    print("x")
```

ii)

```
for i = 0 to n by 3 do
    print("x")
```

iii)

```
for i = 0 to n do
    for j = 0 to m do
        print("x")
```

Note: In the above pseudocode, `for` $i = 0$ `to` n `by` k means we do not increase i by 1, but each time, we increase it by k, i.e., `i=i+k`.

b) Below is a pseudocode of a function named `whatDoesItDo`, which takes an array `A[1..n]` of n integers and an integer k as inputs. Feel free to use intermediate results from **a)**.

```
Algo: whatDoesItDo(A, n, k)
result = −1000
for i = 1 to n do
    current = 0
    for j = i to n by k do
        current = current + A[j]
    if current > result then
        result = current
return result
```

- Perform exact analysis of the running time of the algorithm.
- Determine the asymptotic complexity of the algorithm?

Task 3 (Easy)

Calculate the asymptotic tight bound for the following functions and rank them by their order of growth (lowest first). Clearly work out the calculation step by step in your solution.

$$
\begin{aligned}
f_1(n) &= 10^{\lg 20} n^4 + 8^{229} n^3 + 20^{231} n^2 + 128 n \log n \\
f_2(n) &= \log^2(n) + 50\sqrt{n} + \log(n) \\
f_3(n) &= 14400 \\
f_4(n) &= 2\log(6^{\log n^2}) + \log(\pi n^2) + 4^{log_2 n} \\
f_5(n) &= (2n+3)!
\end{aligned}
$$

Task 4 (Hard)

Consider the algorithm `algo1`. The input parameters are an array `A[1..n]` with n distinct integers and $k \leq n$.

```
Algo: algo1(A, n, k)
---------------------------------
sum = 0;
for i = 1 to k do
    maxi = i;
    for j = i to n do
        if A[j] > A[maxi] then
            maxi = j;
    sum = sum + A[maxi];
    swp = A[i];
    A[i] = A[maxi];
    A[maxi] = swp;
return sum
```

a) In your own words, describe what the algorithm `algo1` does? How do the inner and outer loop contribute to achieving this goal?

b) Based on your description above, formalize the pre/post conditions of the `algo1` algorithm.

c) For the two `for` loops in the algorithm:

   i. Determine if the loop is `up` loop or `down` loop.

   ii. Determine the invariants of these two loops and verify whether they are hold in three stages: `initialization`, `maintenance` and `termination`.

d) Identify some edges cases of the algorithm and verify if the algorithm has the correct output.

e) Conduct an exact analysis of the running time of algorithm `algo1`.

f) Determine the best and the worst case of the algorithm. What is the running time and asymptotic complexity in each case?

Task 5 (Easy)

For each of the statements below determine if they are True or False with explanation:

(1)

Assume $f(n)$ describes the runtime of an algorithm, then $g(n)$ is an asymptotic upper bound for $f(n)$.

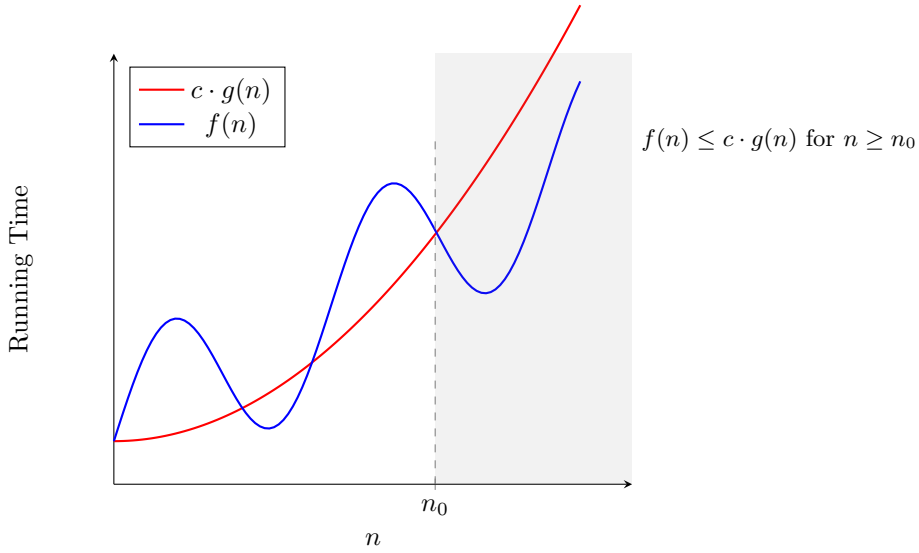


Figure 1: Task 5 (1)

(2)

Each algorithm has at most one upper bound for it's running time.

(3)

If an algorithm has a tight bound it always has the same function as upper and as lower bound.

(4)

The content of a loop with condition **for** *int* $i = n$; $i \geq 1$; $i = i/2$ runs asymptotically $\log_2(n)$ times.